

“던전 (dungeon)” 문제 풀이

작성자: 이종영

부분문제 1

철수의 경로로 가능한 경우의 수는 $\binom{2N-2}{N-1}$ 개가 있으며, 영의의 경로로 가능한 경우의 수도 같다. 따라서 가능한 경로의 조합의 수는 $\binom{2N-2}{N-1}^2$ 개이다. 모든 조합에 대해 실제로 시도해 보며, 각 칸을 지났는지를 $O(N^2)$ 에 확인하면 된다.

부분문제 2

두 경로가 겹치는 부분은 한 행의 연속한 열, 또는 한 열의 연속한 행 뿐이다. 따라서 겹치는 부분으로 가능한 경우의 수는 $O(N^3)$ 개이다.

각 겹치는 부분마다, 해당 경우의 최댓값을 구한다. 이는 각 꼭짓점에서 시작해 해당 칸까지 도달하는 경로의 최대 가치를 $O(N^2)$ 의 동적 계획법으로 전처리한 후, 각 꼭짓점에서 겹치는 부분으로 진입하는 칸들의 조합을 모두 시도해 보면 된다. 이 때, 해당 경로들이 서로 겹치지 않는 경우에만 진입할 수 있음에 유의해야 한다. 한 가지 예시로, 겹치는 부분이 한 행의 연속한 열인 경우를 생각해 보자. 왼쪽 위에서 시작하는 경로는 겹치는 부분의 왼쪽 끝 칸의 윗쪽 또는 왼쪽 칸에 도달할 수 있으며, 왼쪽 아래에서 시작하는 경로는 겹치는 부분의 왼쪽 끝 칸의 왼쪽 또는 아래쪽 칸에 도달할 수 있다. 단, 두 경로 모두 왼쪽 끝 칸의 왼쪽 칸에 도달하는 경우는 제외해야 한다. 겹치는 부분 내의 점수는 부분합 등의 방법으로 $O(1)$ 에 구할 수 있다.

부분문제 3

$N = 2$ 인 경우, 모든 칸을 한 번 이상은 방문하게 되므로 모든 칸의 가치의 합이 답이 된다.

$N > 2$ 인 경우, 겹치는 부분이 두 칸 이상이라면, 적절한 경로의 수정을 통해 정확히 한 칸이 겹치는 경우로 바꿔줄 수 있다. 모든 칸의 가치가 0 이상이므로 이러한 수정을 통해 가치의 합이 감소하지 않음을 알 수 있다. 겹치는 부분이 한 칸인 경우의 수는 $O(N^2)$ 이므로, 모든 경우를 다 시도해보며 서브태스크 2의 풀이를 사용할 수 있다.

부분문제 4

서브태스크 2의 풀이에서 둘 이상의 칸이 겹치는 경우를 최적화한다. 겹치는 부분이 한 행의 연속한 열인 경우를 해결하면, 한 열의 연속한 행인 경우는 격자를 돌려 같은 방식으로 해결할 수 있다.

겹치는 부분의 행을 고정하자. 어떤 열의 구간의 값은 왼쪽 두 꼭짓점에서 오는 경로들이 왼쪽 끝 칸에 어떻게 도달하는지와, 오른쪽 두 꼭짓점에서 오는 경로들이 오른쪽 끝 칸에 어떻게 도달하는지에 따라 정해진다. 구간의 길이가 2 이상이므로 해당 두 값들은 서로 독립이다. 즉, 열의 구간 $[l, r]$ 에 대해, 해당 구간의 값은 $A_l + B_r$ 의 꼴로 나타낼 수 있다. 이제 현재 열 i 를 증가시키며 $l < i$ 인 i 들 중 A_l 의 최댓값을 관리하면 $A_l + B_i$ 가 답의 후보가 된다. 따라서 각 행에 대해 $O(N)$ 에 문제를 해결할 수 있고, 전체 문제가 $O(N^2)$ 에 해결된다.

“외곽 순환 도로 2 (ringroad2)” 문제 풀이

작성자: 구재현

부분문제 1

모든 홀수 사이클과 교집합이 있는 간선 집합이라는 것은, 해당 간선 집합을 제거하였을 때 남은 그래프가 이분 그래프 (Bipartite Graph) 라는 것이다. 이분 그래프의 경우 2-Coloring이 존재한다. 다시 말해, 모든 간선의 양 끝점이 서로 다르도록 정점의 색을 **흰색** 과 **검은색** 으로 칠할 수 있다는 것이다.

따라서, 간선 집합을 열거하는 대신, 정점의 색을 모두 열거한 후 **같은 색을 잇는 간선의 가중치 합** 을 최소화하는 식으로 문제를 변형할 수 있다. 이렇게 변형한다고 문제가 크게 달라지지는 않지만, 이 구조가 DP를 적용하기 가장 적절하다.

서브태스크 1의 경우, 외곽 순환 도로를 이루는 정점의 수가 작다. 이 정점들의 색의 경우는 2^K 가지이다. 이 경우의 수를 모두 열거하자. 이 경우 외곽 순환 도로들의 양 끝점 색이 고정되니, 가중치를 더하거나 더하지 않은 후 제거할 수 있다. 남은 그래프는 트리가 된다.

이제 다음 문제를 풀면 된다:

- 트리가 주어지고, 일부 정점에 색이 고정되어 있을 때, 모든 정점에 흰색/검은색 을 부여해서, 같은 색을 잇는 간선의 가중치 합을 최소화하라.

이는 $DP[v][c] = (v \text{의 서브트리에서 모든 색을 칠했고, } v \text{의 색은 } c \text{ 일 때, 서브트리에서 필요한 비용})$ 이라는 DP를 정의하면 해결할 수 있다. 상태 전이는 간단하다.

트리 DP를 선형 시간에 해결할 수 있고, 이를 2^K 번 해결하니, 시간 복잡도는 $O(2^K N)$ 이다.

부분문제 2

부분 문제 1의 관찰을 사용하자. 일반성을 잃지 않고 0 번 점이 흰색이라고 하면, 사이클 상 색의 모든 조합을 DP로 열거한 후 이에 따른 비용을 계산할 수 있다. 부분 문제 1의 관찰이 없이도 유사한 DP를 얻을 수 있으나 정의나 상태 전이가 더 복잡할 것이다. 시간 복잡도는 $O(N)$ 이다.

부분문제 3

외곽 순환 도로를 끊을 수 없기 때문에, 순환 도로 상 리프의 색이 모두 고정된다. 부분 문제 1과 동일하게 해결하면 된다. 시간 복잡도는 $O(N)$ 이다.

부분문제 4

부분 문제 3과 다른 점은 K 가 홀수일 경우 외곽 순환 도로 중 하나를 끊어야 한다는 것이다. 리프의 색상 조합이 $O(N)$ 개 생기는데, 이는 모두 열거해 보기엔 많다. 부분 문제 3의 DP를 수정하여, 서브트리가 끊겨진 외곽 순환 도로의 왼쪽에 있는지, 오른쪽에 있는지, 혹은 그 사이인지를 인자로 저장한다. 리프의 색은 외곽 순환 도로 상 위치, 그리고 위 인자에 따라 결정된다. 이 경우 모든 리프의 색상 조합을 열거함과 동시에 최적해를 구할 수 있다. 시간 복잡도는 $O(N)$ 이다.

부분문제 6

부분문제 1과 비슷하게 트리 DP로 해결하는데, 외곽 순환 도로에 대한 고려가 필요하다. 이를 고려해 주기 위해서, 다음과 같이 DP 상태를 정의하자:

- $DP[v][c][l][r] = (v \text{의 서브트리에 모든 색을 칠했고, } v \text{의 색은 } c \text{이고, } v \text{의 서브트리에서 왼쪽으로 나가는 외곽 순환 도로와 인접한 정점의 색이 } l, \text{ 오른쪽으로 나가는 외곽 순환 도로와 인접한 색이 } r \text{이다.})$

0번 정점을 루트로 해서 트리를 구성하면, 서브트리 안과 밖을 잇는 외곽 순환 도로는 정확히 두 개가 될 것이다. 이 두 외곽 순환 도로의 서브트리 안쪽 정점 색을 저장한다고 보면 된다.

각 자식에 대한 DP 값을 재귀적으로 계산한 이후, 상태 전이를 진행한다. 두 서브트리를 합쳐줄 때, 두 서브트리 사이를 잇는 외곽 순환 도로가 다른 색의 정점을 잇는지, 같은 색의 도로를 잇는지에 따라서 가중치가 추가될 수도 있고 아닐 수도 있다. 예를 들어, $DP[v_1][c][l_1][r_1]$ 와 $DP[v_2][c][l_2][r_2]$ 를 합친다고 하면, $r_1 = l_2$ 일 경우 비용을 지불해야 할 것이고, 합친 결과는 $DP[v][c][l_1][r_2]$ 와 같을 것이다.

이러한 식으로 상태 전이를 고려해 주면, 각 정점당 $O(1)$ 시간에 DP 테이블을 합칠 수 있다.

매 합치는 과정에서 외곽 순환 도로의 비용을 찾는 것이 어려울 수 있는데, 만약 위 과정을 일반적인 DFS로 하게 된다면, DFS를 하는 과정에서 외곽 순환 도로의 비용을 preorder 순으로 읽기 때문에, 단순히 큐와 비슷하게 필요할 때 맨 앞에 있는 Weight를 뺄 수 있다. 이러한 방식으로 구현하면 코드가 아주 짧으나, 그 외 다른 방식으로 구현하여도 크게 어렵지 않다.

시간 복잡도는 $O(N)$ 이다.

여담

위와 같은 문제를 Odd-Cycle (Edge) Transversal 이라고 하며, 2020 선발고사의 아이싱 문제도 이러한 류의 문제에 속한다. Treewidth가 $tw(G)$ 인 그래프에 대해서, Odd-Cycle Edge Transversal은 $O(4^{tw(G)}N)$ 에 트리 DP로 해결되니, Odd-Cycle Edge Transversal은 Treewidth에 Fixed-Parameter Tractable하다.

이 문제에서 다룬 그래프는 $tw(G) \leq 3$ 이기 때문에 위 풀이를 적용할 수 있으며, 실제로 이러한 코드로도 만점을 받을 수 있다. 위에서 다룬 풀이는 그래프의 Ad-hoc한 성질을 더 활용하여 간소화한 풀이라고 할 수 있다.

“택시 (taxi)” 문제 풀이

작성자: 윤창기

부분문제 1

$N \leq 20$ 으로 지수 시간 알고리즘이 동작하는 제한이다. 가장 마지막으로 방문한 도시와, 지금까지 방문한 도시의 집합을 저장하는 동적 계획법으로 해결할 수 있다.

부분문제 3

$N \leq 2000$ 인 경우로, 임의의 두 점 i, j 에 대해 i 에서 j 로 가는 $B_i \text{dist}(i, j)$ 비용의 간선을 이어주면 Dijkstra's algorithm 등을 통해 $O(N^2)$ 시간 안에 해결할 수 있다. 여기서 $\text{dist}(i, j)$ 는 입력에 주어진 트리에서 두 점을 잇는 최단경로의 길이로 정의한다. 이후에는 Dijkstra's algorithm을 보다 효율적으로 subquadratic 시간에 해결하는 방법을 찾아야 한다.

부분문제 2

주어진 트리가 직선인 경우, Dijkstra's algorithm을 보다 최적화한 동적 계획법을 생각할 수 있다. D_i 를 1번 도시에서 i 번 도시로 이동하는 최소 비용으로 정의하면, $D_1 = 0$, $i \neq 1$ 에 대해서는 다음이 성립한다.

$$\begin{aligned} D_i &= \min_{j \neq i} D_j + B_j |C_i - C_j| \\ &= \min \left(\min_{j < i} [D_j - C_j B_j + B_j C_i], \min_{j > i} [D_j + C_j B_j - B_j C_i] \right) \end{aligned}$$

여기서 $C_i = \text{dist}(1, i)$ 로 정의한다. 이 때, i 에서 j 로 이동하는 데 이어 j 에서 k 로 이동한다면, $B_i > B_j$ 가 성립한다고 가정해도 좋다. 그렇지 않은 경우 i 에서 k 로 직접 이동하는 것에 비해 비용이 더 낮을 수 없기 때문이다. 따라서 $1, \dots, n$ 의 모든 정점을 B_i 가 내림차순이 되는 순서로 정렬하고, 위 동적 계획법 식을 볼록 껍질 최적화 (Convex Hull Optimization) 등을 통해 $O(N \log N)$ 시간에 해결할 수 있다.

부분문제 4

$B_i \leq 30$ 이 성립하는 경우이다. 위의 경우에 적용했던 동적 계획법을 트리에 대해 다시 쓰면 아래와 같다. 편의상 $B_i < B_1$ 인 정점 i 에 대해서만 생각하자. 그렇지 않은 정점들에 대해서는 답이 자명하다.

$$D_i = \min_{B_j > B_i} D_j + B_j \cdot \text{dist}(i, j) \quad (1)$$

고정된 b 와 $B_i = b$ 를 만족하는 정점 i 의 목록, 그러한 정점 i 의 목록이 주어진 경우 이 정점들에 의한 식 ??의 갱신을 Multi-source Dijkstra's algorithm을 활용하여 한꺼번에 $O(N \log N)$ 에 수행할 수 있다. 따라서 b 를 내림차순으로 순회하며 이를 반복하면 전체 문제를 $O(\max B_i \cdot N \log N)$ 시간에 해결할 수 있다.

부분문제 5

$B_i > 0$ 을 만족하는 정점의 수 K 가 적은 경우이다. $B_i > 0$ 인 정점의 택시만 이용하는 답은 부분문제 3과 같이 Dijkstra's algorithm을 활용하여 $O(K^2)$ 시간 안에 구할 수 있다. $B_i = 0$ 인 정점에 도착하는 최소 비용은 트리를 여러 직선으로 분할하는 Heavy-Light Decomposition 등을 이용하여 계산할 수 있다. $B_i > 0$

인 정점의 조상에 위치한 Heavy Chain은 최대 $\log N$ 개 존재할 수 있으므로, Heavy Chain h 중 i 에 가장 가까운 정점 $h(i)$ 에 초기 비용 $D_i + B_i \text{dist}(h(i), i)$, 거리당 비용 B_i 짜리 택시가 있다고 가정해도 좋다. 이와 같이 heavy chain에 등록되는 새로운 정점의 개수가 최대 $K \log N$ 개이고, 부분문제 2와 같이 Convex Hull Optimization을 이용하여 $K^2 + N \log N$ 시간에 문제를 해결할 수 있다.

부분문제 6

추가 제약 조건이 없는 경우로, 전체 문제를 해결하기 위해서는 B_i 가 감소하는 순서대로 탐색하면 된다는 사실과 함께 Centroid Decomposition 등의 자료구조를 활용할 수 있다.

정점들을 B_i 가 감소하는 순서로 정렬한다. 어떤 정점 i 에 대해 D_i 가 결정되어, 다른 점으로 이동하는 경우를 생각해보자. 만약 i 가 트리의 루트라면, 다른 모든 정점 j 에 대해 $D_j \leftarrow \min(D_j, D_i + B_i \text{depth}(j))$ 와 같이 갱신하는 것으로 충분하다. 이는 y 절편이 D_i , 기울기가 B_i 인 직선을 $x = \text{depth}(j)$ 에서 평가한 것으로 생각할 수 있다.

실제로는 i 가 루트가 아니므로, i 의 모든 부모 p 에 대해 p 의 서브트리에 속하는 정점 v 를 잡고 $D_v \leftarrow \min(D_v, D_i + B_i(\text{depth}(i) + \text{depth}(v) - \text{depth}(p)))$ 와 같이 갱신해줄 수 있다. 이는 y 절편이 $D_i + B_i(\text{depth}(v) - \text{depth}(p))$, 기울기가 B_i 인 직선으로 생각할 수 있다.

어떤 D_j 를 계산하기 위해서는 과거에 생성된 직선들을 $x = \text{depth}(j)$ 에서 계산한 값 중 최솟값을 찾음으로써 계산할 수 있고, Convex Hull Optimization을 사용하면 직선의 개수가 L 개일 때 총 $O(L \log L)$ 시간 내에 새로운 직선을 삽입하거나, 임의의 점에서 직선들 중 최솟값을 찾는 자료구조를 지원할 수 있다.

단순한 방법으로 이를 구현할 경우 직선이 총 $O(N^2)$ 개가 되므로, 이를 원래 트리가 아닌 Centroid Decomposition 과정을 나타낸 트리에서 수행하는 경우 생성되는 직선의 개수를 $O(N \log N)$ 개로 줄일 수 있으며, 전체 문제를 $O(N \log^2 N)$ 시간에 해결할 수 있다.

“야유회 (workshop)” 문제 풀이

작성자: 구재현

부분문제 1

```
return my_num;
```

17점 ($m = 34$)

afternoon, evening 함수는 return my_num; 을 하게 하고, morning 함수만 사용한다.

모든 $0 \leq u, v, w \leq N - 1, u \neq v, v \neq w$ 에 대해서 $f(u, v) \neq f(v, w)$ 를 만족하며 반환값이 충분히 작은 함수 f 를 구성하자. 이러한 함수가 있으면, morning 함수는 단순히 $f(my_num, right_num)$ 을 반환하면 된다.

u 와 v 를 이진 비트열이라고 생각하자. $u \neq v$ 이기 때문에 둘이 다른 값을 가지는 비트가 존재할 것이다. $u[k] \neq v[k]$ 라고 할 경우, $f(u, v) = 2k + u[k]$ 라고 정의하자. $f(u, v) = f(v, w)$ 일 경우, $u[k] = v[k]$ 가 만족해야 하는데, 이는 가정에 모순이다.

이렇게 구성한 $f(u, v)$ 는 최대 $2\lceil \log_2 N \rceil$ 의 크기를 가진다. 입력 제한에서 이는 최대 34 이며 16점을 얻는다.

46점 ($m = 8$)

afternoon, evening 함수의 left_num 인자를 무시하자. 17점의 방법을 나머지 두 함수에 그대로 적용하면, $34 \rightarrow 12, 12 \rightarrow 8$ 로 크기를 줄일 수 있다.

57점 ($m = 6$)

afternoon, evening 함수의 left_num 인자를 활용해서 크기를 한번 더 줄일 수 있다. $comp(u, v, T)$ 를, $0 \leq u, v \leq 2^T - 1, u \neq v$ 인 두 정수가 주어졌을 때 $[0, 2^T - 1]$ 이하의 정수를 반환하는 함수라고 하자 ($f(u, v)$ 의 정의를 약간 변형하였다). morning 함수에서 크기를 34 이하로 줄였다면,

- $afternoon(l, x, r) = comp(comp(l, x, 6), comp(x, r, 6), 4)$
- $evening(l, x, r) = comp(x, r, 3)$

와 같다. 위의 풀이에서 항상 인접한 수가 다르다는 불변량을 유지하고 있기 때문에, 인자가 3개 들어오는 경우 압축을 두 번 수행할 수 있다.

$comp$ 함수는 $[0, 5]$ 범위의 정수를 압축할 수 없기 때문에, evening 함수에서는 left_num 인자를 참조하여도 도움이 되지 않는다.

81점 ($m = 4$)

현재의 $comp(u, v, T)$ 는 $[0, 2^T - 1]$ 의 수를 받아 $[0, 2^T - 1]$ 의 수를 반환하는데, 이 때문에 6 미만의 압축이 불가능하다는 한계가 있다. 함수의 정의를 조금 바꾸면 4 까지 압축이 가능하다.

길이 $2T$ 의 이진 문자열 중 0 이 T 개이고 1 이 T 개인 문자는 $\binom{2T}{T}$ 개이다. $0 \leq u, v \leq \binom{2T}{T} - 1$ 인 두 수 u, v 에 대해서, 이 두 수를 위 성질을 만족하는 서로 다른 이진 문자열로 매핑하자. 1 의 개수가 고정되어

있기 때문에, $u[k] = 0, v[k] = 1$ 인 k 가 무조건 존재한다. 또한, $f(u, v) \neq f(v, w)$ 가 성립할 수밖에 없다 ($v[k] = 1$). 이를 사용하면, $\text{comp}(u, v, T)$ 의 입력 크기 제한을 $[0, 2^T - 1]$ 에서 $[0, \binom{2^T}{T} - 1]$ 로 제공에 가깝게 늘릴 수 있다. 이제

- $\text{morning}(x, r) = \text{comp}(x, r, 10)$
- $\text{afternoon}(l, x, r) = \text{comp}(\text{comp}(l, x, 3), \text{comp}(x, r, 3), 2)$
- $\text{evening}(l, x, r) = x$

로 두면 된다 ($\binom{20}{10} \geq 180\,000$, $\binom{6}{3} = 20$, $\binom{4}{2} = 6$).

이 함수 역시 4 미만의 압축을 할 수는 없기 때문에, **evening** 함수는 사용하지 않는다.

이진 문자열로의 매핑은, **init** 함수에서 모든 $\binom{2^T}{T}$ 개의 조합을 단순히 다 나열하는 것으로 쉽게 구현할 수 있다.

만점 ($m = 3$)

evening 함수에서 $[0, 3]$ 범위의 세 정수 l, x, r 이 주어졌을 때, 이를 $[0, 2]$ 범위의 정수로 매핑해야 한다. 이는 크게 두 가지 방법으로 해결할 수 있다.

가장 간단한 방법은 다음과 같다. 만약 $x \leq 2$ 일 경우, x 를 그대로 반환해 준다. $x = 3$ 일 경우, $l, r \leq 2$ 이기 때문에 이 두 수가 바뀌지 않음이 보장된다. $\{0, 1, 2\}$ 중 $\{l, r\}$ 중 어느 것도 아닌 수가 무조건 하나 존재하기 때문에, x 를 이 수로 바꿔도 인접한 수가 다르다는 조건이 유지된다.

그 외, 위 문제를 36 개의 정점을 가진 그래프의 3-coloring 문제로 변환한 후, 이 값을 로컬에서 백트래킹으로 찾고, 백트래킹 결과를 하드코딩하는 방법도 있다. 잘 구현된 백트래킹은 순식간에 3-coloring을 찾을 수 있다.

여담

이 문제는 Distributed Computing 환경에서 Graph Coloring 문제를 해결하는 것과 밀접한 연관성이 있다. 위 문제의 세팅은 사이클의 노드의 값이 서로 다른 parallel machine에 저장되어 있을 때, $O(\log^*(n))$ 번의 Iteration 만을 사용하여 사이클의 3-coloring을 찾는 것이라고 생각할 수 있다. 여기서 $\log^*(n)$ 은, $n \leftarrow \log_2 n$ 을 반복했을 때 $n \leq 1$ 이 되게끔 하는 최소 반복 수를 뜻한다.

일반적인 그래프에서도 위의 접근을 확장시켜, 그래프의 최대 차수를 Δ 라고 할 때 $(\Delta + 1)$ -coloring을 $O(\log^*(n) + \Delta)$ 시간에 찾을 수 있다. 간단히 요약하자면, 위 풀이는 Functional graph에서도 동일하게 적용할 수 있고, 그래프를 Δ 개의 Functional graph로 분해한 후 각각에 대해 위와 같은 알고리즘을 동일하게 적용한 후 이를 합쳐줄 수 있다. 이러한 알고리즘을 *Cole-Vishkin Algorithm* 이라고 한다.

관련된 내용을 더 탐구해 보고 싶으면 <https://jukkasuomela.fi/dda-2010/lecture-2.pdf> 을 읽어보면 좋다.